Count Fuzzer labjournal

Anna Latour

2022

Research goals

- Develop fuzzer for counting problems, including support for:
  - Exact model counting.
  - PAC counting.
  - Projected counting.
  - Weighted model counting.
- Develop delta debugger for counting problems, inlcuding support for the problems mentioned above.
- Identify problems of the following kinds in existing counters:
  - Errors (unexpected termination without providing a result).
  - Incorrect counts (we might have to define that for PAC counters).
- Fuzz the preprocessor(s).
- Generate instances with known ground truth count, especially for weighed model counting.

Research questions and design questions

- How to automatically generate diverse problem instances for the different counting problems?
- How to automatically generate challenging problem instances for the different counting problems?
- How easy/hard should these problems be?
- How do we determine the true count?
- How do we define an incorrect count in the case of PAC counting?
- (How) do we account for differences in interpretation of the contribution to the model count of free variables?
- How to efficiently minimise the instance size of bug triggering instances?

- How modular should the resulting tool be? There are many different encodings (projected or not, but also different ways of encoding Bayesian networks into CNF) that are not all compatible with all solvers. Does it make sense to have functionality to create problems of a certain type (*e.g.*, Baysian network) and let the user implement their favourite encoding?

Experimental evaluation

We have four main experiments to perform.

Counting errors, incorrect counts, and maybe wrong answers

The first measures how many errors and incorrect counts we find for each solver. Following [Brummayer et al., 2010][1] we should let the fuzzer generate a fixed number of instances (*e.g.*, 10 000) and see how many of those lead to errors and incorrect counts in the solvers on which we evaluate. Let's call this set of instances $\mathcal{T}$.

It is possible that the same bug can lead to an error in one instance, and an incorrect count in another. It is also possible that multiple instances trigger the same bug. To gain more insight in this dynamic, we can run a second experiment.

We can take a solver and solve the errors and miscounts. Then, we can for each bugfix record how many errors and miscounts are resolved by that bugfix. That way, we'll have some idea of how much overlap there is in the instances that trigger the bugs, and in the outcome of those bugs (errors and miscounts).

[1] R. Brummayer, F. Lonsing, and A. Biere. Automated testing and debugging of SAT and QBF solvers. In O. Strichman and S. Szeider, editors, *Theory and Applications of Satisfiability Testing - SAT 2010, 13th International Conference, SAT 2010, Edinburgh, UK, July 11-14, 2010. Proceedings*, volume 6175 of *Lecture Notes in Computer Science*, pages 44–57. Springer, 2010. DOI: 10.1007/978-3-642-14186-7_6. URL https://doi.org/10.1007/978-3-642-14186-7_6

Comparison with CNFuzz

We should also run an experiment in which we evaluate how many bugs are triggered by instances generated by CNFuzz, to demonstrate the added value of our system(s).

Case study: ApproxMC4

For example, we could take ApproxMC4 [Chakraborty et al., 2016, Soos and Meel, 2019][2] as this solver. We would then take the most recent commit from *before* we started this project as the baseline version of ApproxMC4 (here we would have to take care to also use the appropriate commits for CryptoMiniSAT [Soos et al., 2009][3] and Arjun [Soos and Meel, 2021][4]). While we develop the fuzzer, we will undoubtedly find errors and miscounts in ApproxMC4. We should

[2] S. Chakraborty, K. S. Meel, and M. Y. Vardi. Algorithmic improvements in approximate counting for probabilistic inference: From linear to logarithmic SAT calls. In *Proceedings of International Joint Conference on Artificial Intelligence (IJCAI)*, 7 2016; and M. Soos and K. S. Meel. BIRD: Engineering an efficient CNF-XOR SAT solver and its applications to approximate model counting. In *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*, 1 2019

[3] M. Soos, K. Nohl, and C. Castelluccia. Extending SAT solvers to cryptographic problems. In O. Kull-

properly record the instances that triggerd them, and the commits in which the bugfixes were implemented. After we have run the "pre-project commit" version of ApproxMC4 on the instances in $\mathcal{T}$ for the experimental evaluation, we can run whatever version of ApproxMC4 we have by then on $\mathcal{T}$, and fix whichever bugs arise from there, again carefully recording the instances and the commits with the bugfixes.

Finally, we can then classify all the bugfixes to determine if they fix an error or a miscount, and link them to the instance(s) from $\mathcal{T}$ that triggered them. From there, we can do some reporting on how many instances trigger the same bug, and if bugfixes tend to repair solely errors or miscounts or both.

Delta-debugging

The fourth main experiment would evaluate the effectiveness of the delta debugger. Following [Brummayer et al., 2010], we can take the instances from $\mathcal{T}$ that trigger errors or miscounts, minimise them, and record the time it takes to minimise them and record by how much we could minimise them.

Relevant Literature

- [Zeller, 2009] *Why Programs Fail - A Guide to Systematic Debugging, 2nd Edition*.
- [Brummayer and Biere, 2009] *Fuzzing and Delta-Debugging SMT Solvers*.
- [Brummayer et al., 2010] *Automated Testing and Debugging of SAT and QBF Solvers*.
- [Fichte et al., 2022] *Proofs for Propositional Model Counting*.

Counters to evaluate

We may want to focus on counters that are at most 5 years old and/or are still maintained.

It makes sense to focus on the counters from the last competition(s), and to use the default parameters with which they were entered in those competitions.

Exact counters

- DPMC.[5]
- cachet [Sang et al., 2005a].
- GANAK [Sharma et al., 2019].
- GPMC.[6]
- ExactMC [Lai et al., 2021].
- SharpSAT [Thurley, 2006].
- SharpSAT-TD.[7]

[5] Available at github.com/vardigroup/DPMC.

[6] Available at www.trs.cm.is.nagoya-u.ac.jp/projects/PMC.

[7] Available at github.com/Laakeri/sharpsat-td.

Compilers

- C2D [Darwiche, 2004] CNF-to-d-DNNF compiler. *Binary only!*[8]
- cnf2eadt [Koriche et al., 2013] CNF-to-EADT compiler. *Binary only!*[9]
- CNF2OBDD [Toda and Soh, 2016]. CNF-to-OBDD compiler based on miniSAT.[10]
- D4 CNF-to-Decision-DNNF compiler. *Binary only!*.[11]
- DSHARP [Muise et al., 2012] CNF-to-d-DNFF compiler.[12]
- miniC2D [Oztok and Darwiche, 2015] Compilation-based, DPLL-based CNF-to-SDD compiler.[13]
- SDD [Choi and Darwiche, 2013] CNF-to-SDD compiler.[14]
- Tree-of-BDDs [Subbarayan et al., 2007] CNF-to-tree-of-BDDs compiler. *Cannot find code or binary.*

[8] Available at http://reasoning.cs.ucla.edu/c2d.

[9] Available at www.cril.univ-artois.fr/KC/eadt.html.

[10] Available at www.sd.is.uec.ac.jp/toda/code/cnf2obdd.html.

[11] Available at github.com/crillab/d4.

[12] Available at github.com/QuMuLab/dsharp.

[13] Available at reasoning.cs.ucla.edu/minic2d.

[14] Available at reasoning.cs.ucla.edu/sdd.

Approximate counters

- ApproxCount Local search algorithm [Wei and Selman, 2005].
- ApproxMC4 Sampling algorithm with PAC-guarantees [Chakraborty et al., 2016, Soos and Meel, 2019, Soos et al., 2020].
- FocusedFlatSAT (by Stefano Ermon, 2011).[15]
- MBound (by Ashish Sabharwal, Cornell University, 2007).[16]
- SampleCount [Gomes et al., 2007].[17]
- Search Tree Sampler [Ermon et al., 2012].[18]

[15] Available at cs.stanford.edu/ ermon/code/FocusedFlatSAT.zip.

[16] Available at www.cs.cornell.edu/ sabhar/software/xor-scripts/xor-scripts.zip.

[17] Available at www.cs.cornell.edu/ sabhar/software/samplecount/sa 1.0-04092007.zip.

[18] Available at cs.stanford.edu/ ermon/code/STS.zip.

Weighted counters

- ADDMC [Dudek et al., 2020].[19]
- ApproxMC-p Projected weighted model counting [Klebanov et al., 2016].[20]
- cachet-wmc [Sang et al., 2005b].
- GANAK has projected(?) weighted model counting functionality [Sharma et al., 2019].
- WeightMC Approximate weighted model counter [Chakraborty et al., 2014].[21]
- weighted sharpSAT.[22]

[19] Available at github.com/vardigroup/ADDMC.

[20] Available at formal.kastel.kit.edu/ weigl/software/approxmc-py.

[21] Available at meel-group.github.io/software/weightmc.

[22] Available at bitbucket.org/latower/weighted-sharpsat/src/master.

Preprocessors

- Arjun [Soos and Meel, 2021].
- B+E [Lagniez et al., 2016, 2020].

READING

Summary of [Brummayer and Biere, 2009]

Title: *Fuzzing and Delta-Debugging SMT Solvers*
Reading date: 2022-10-31

The authors present fuzzers and delta-debuggers for CNF and QBF formulae.

Specifically, they present three fuzzers based on *grammar-based black-box fuzzing*: CNFuzz, FuzzSAT, BlocksQBF and QBFuzz.

While the authors had access to a random 3SAT generator (3SAT-Gen), they decided to create CNFuzz. This random instance generator incorporates structure in the generated CNFs, which better reflects reality and has a better chance of triggering bugs in the SAT solvers, because they heavily rely on structure for solving the instances. The FuzzSAT instance generator is event more structured, translating random DAGs into CNF.

The authors also implemented BlocksQBF, which generates random QBFs based on a model from the literature that they adapted for this purpose. To increase the diversity of the generated instances, they also implement QBFuzz, which has a different approach to selecting literals to incorporate in clauses.

The authors present experiments where they take the three different SAT fuzzers, have them each generate 10 000 instances, and record the numbers of *errors* (early termination with no result), *incorrects* (solver reporting that CNF is UNSAT while it is SAT or vice versa),[R1] and *invalid models* (where the solver correctly identifies a CNF as SAT, but returns an assignment that is not in fact a model),[R2] for different solvers.

[R1] The equivalent for counting would be a model count of 0 if a formula is SAT.

[R2] The equivalent for model counting would be an incorrect count.

They perform similar experiments for the two different QBF fuzzers.

Their other contribution is on delta debugging. They implement two delta debuggers for SAT, `cnfdd` and the multithreaded version `mtcnfdd`, and one delta debugger for QBF, `qbfdd`. Then, they took the generated instances that triggered bugs in the solvers, and used the delta debuggers to minimise those instances to the smallest size where they would still trigger the bugs on those solvers.[Q1] They found that they can often reduce the size by over 85%, the only exception being a SAT solver that contained bugs that only showed up after many propagations, and hence needed somewhat larger formulae. Instance size was measured in file size. They also report minimisation time.

[Q1] What is their minimisation algorithm?

JOURNAL

Mon 31 Oct 2022: Anna & Mate

Brainstorm about scope of the project, instigated by the question of how we should keep track of the bugs that we identify.

Question: what could be our scientific contribution?

[Brummayer et al., 2010]'s contribution was in part in the different CNF generators that they built for fuzzing SAT solvers and QBF solvers. We could do something similar for model counting.

At first glance, it seems that we should be able to simply reuse the CNF generators that [Brummayer et al., 2010] built. However, there are some subtleties in our case.

First of all, we need a ground truth. With SAT solving, one can verify the correctness of a returned model. For model counting, the counts are harder to verify. For weighted model counting in particular, we als have to deal with the precision of the floats used in the counter.

Hence, we see a possible scientific contribution in finding an intelligent way of generating diverse test instances for (weighted) model counting.

One approach could be to encode *Bayesian networks (BNs)* into

CNF, and use a python package or something to obtain ground truths on the probabilities that we encode. We could add some diversity by encoding them using different encoding schemes. Here are a few:

Dar02 Intuitive, straightforward approach proposed in [Darwiche, 2002][23]. For each variable, the weights of the positive and the negative literal add up to 1.

SBK05 Proposed by [Sang et al., 2005b][24] as more compact alternative to **Dar02**. Less intuitive, but literal weights also add up to 1.

CD05 Proposed by [Chavira and Darwiche, 2005][25] as an encoding to take advantage of local structure in the BN. It also uses shared variables. The idea is to have a single variable for each weight in the problem, to limit the size of the encoding. Additionally, [Chavira and Darwiche, 2005] implements an optimisation in which it uses clauses where exactly one literal must be true. The consequence, however, is that this encoding is not compatible with standard DPLL-based counters.

CD06 Proposed by [Chavira and Darwiche, 2006][26] as an encoding that should work particularly well for DPLL-based solvers, since it facilitates the discovery of components. Uses prime implicants to minimise the encoding. Since this encoding is based on the **CD05** encoding, it is also not compatible with standard DPLL solvers.

BKLM16 This encoding is proposed by [Bart et al., 2016][27] and is based on the **CD06** encoding. However: they made it compatible with standard DPLL-based solvers.

We are currently not aware of any other encodings, but should check the most recent literature.

ADDENDUM (2 nov 2022): *I just found that [Dilkas and Belle, 2021b][28] has a nice summary of the properties of these encodings, and proposes a new one. [Dilkas and Belle, 2021a][29] shows how to remove variables and clauses from existing encodings of Bayesian networks, to make them more compact.*

We could generate different Bayesian networks, encode them with different encodings, and make sure that the weights are diverse too (including extremely high and low weights), to challenge the counters.

We could also think of other ways of generating CNFs with a known model count, or even of new encodings (although that seems a bit out of scope?).

[23] A. Darwiche. A logical approach to factoring belief networks. In D. Fensel, F. Giunchiglia, D. L. McGuinness, and M. Williams, editors, *Proceedings of the Eights International Conference on Principles and Knowledge Representation and Reasoning (KR-02), Toulouse, France, April 22-25, 2002*, pages 409–420. Morgan Kaufmann, 2002

[24] T. Sang, P. Beame, and H. A. Kautz. Performing Bayesian inference by weighted model counting. In M. M. Veloso and S. Kambhampati, editors, *Proceedings, The Twentieth National Conference on Artificial Intelligence and the Seventeenth Innovative Applications of Artificial Intelligence Conference, July 9-13, 2005, Pittsburgh, Pennsylvania, USA*, pages 475–482. AAAI Press / The MIT Press, 2005b. URL http://www.aaai.org/Library/AAAI/2005/aaai05-075.php

[25] M. Chavira and A. Darwiche. Compiling Bayesian networks with local structure. In L. P. Kaelbling and A. Saffiotti, editors, *IJCAI-05, Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence, Edinburgh, Scotland, UK, July 30 - August 5, 2005*, pages 1306–1312. Professional Book Center, 2005. URL http://ijcai.org/Proceedings/05/Papers/0931.pdf

[26] M. Chavira and A. Darwiche. Encoding CNFs to empower component analysis. In A. Biere and C. P. Gomes, editors, *Theory and Applications of Satisfiability Testing - SAT 2006, 9th International Conference, Seattle, WA, USA, August 12-15, 2006, Proceedings*, volume 4121 of *Lecture Notes in Computer Science*, pages 61–74. Springer, 2006. DOI: 10.1007/11814948_9. URL https://doi.org/10.1007/11814948_9

[27] A. Bart, F. Koriche, J. Lagniez, and P. Marquis. An improved CNF encoding scheme for probabilistic inference. In G. A. Kaminka, M. Fox, P. Bouquet, E. Hüllermeier, V. Dignum, F. Dignum, and F. van Harmelen, editors, *ECAI 2016 - 22nd European Conference on Artificial Intelligence, 29 August-2 September 2016, The Hague, The Netherlands - Including Prestigious Applications of Artificial Intelligence (PAIS 2016)*, volume 285 of *Frontiers in Artificial Intelligence and Applications*, pages 613–621. IOS Press, 2016. DOI: 10.3233/978-1-61499-672-9-613. URL https://doi.org/10.3233/978-1-61499-672-9-613

[28] P. Dilkas and V. Belle. Weighted model counting with conditional weights for Bayesian networks. In C. P. de Campos, M. H. Maathuis, and E. Quaeghebeur, editors, *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence, UAI 2021, Virtual Event, 27-30 July*

Mon 31 Oct 2022: Anna's afterthoughts

We could try to use a *binary decision diagram (BDD)* library to help us construct problem instances. The idea would be to build a CNF by generating (random or somehow structured) clauses, and adding them to the formula one by one. We would construct the BDD while we are doing it. BDD libraries like (the Python interface of) CUDD provide functionality for retrieving the CNF afterwards.[30] As we are building the BDD incrementally, we can keep track of the model count of the CNF that it represents, by doing upward counting sweeps of the BDD. That way, we can steer towards, *e.g.*, CNFs with a very small model count compared to its number of clauses or its number of variables.

[30] See web.mit.edu/sage/export/tmp/y/usr/share/doc/poly

Whichever way we choose, the correctness of our count would also depend on the correctness of our encoding. Hence, we may be a snake eating our own tail here?

Wed 2 Nov 2022: Feedback from group

I shared some of the ideas that we had with the group (Mahi, Tim, Arijit, Jiong and Pang) and asked them what they thought of the ideas and if they saw a potentially valuable scientific contribution.

The concensus was that if we come up with a good definition of 'diversity' in the generated instances, and build an instance generator that displays that diversity *and* generates instances with a known ground truth, then that would be a valuable scientific contribution.

Mahi suggests to use ASP to enumerate subgraphs and Hamiltonian paths in graphs, and use that as a basis for formulas with a ground truth.

Arijit suggests that we look at the fuzzer Armin built for Cadical.

Nobody seems against the idea of using Bayesian networks or SDDs/BDDs/other DDs to generate instances, *on the condition* that the generator is verified. So that'll be fun.

Another idea that came up to deal with fuzzing ApproxMC4 is the following. Don't record the number of wrong counts, but only record the errors (early termination without output) and the incorrect answers (model count of 0 while satisfiable, or non-zero model count when unsatisfiable). Take the 'original' version of ApproxMC4 (before debugging), and to run it for 10 different random seeds on each instance, and recording the distribution of the counts for all instances that don't result in an error or incorrect answer. Then, on the debugged version (after debugging such that none of instances

generated by the fuzzer triggers a bug), do the same thing. Then compare the distributions. If there were bugs that caused wrong counts, one would expect to see that reflected in the distributions of the counts.

Another idea that came up is to define diversity in terms of different features, like:

- Number of variables/literals/clauses.
- Ratios of the above.
- Mean/median/max clause length.
- Tree width / diameter / average clustering coefficient of primal graph.
- Max clique size, number of certain motifs etc of primal graph.

Thu 3 Nov 2022: Anna & Mate

We talked about instance generation and ground truths. Long story short: providing certified proof of a count is going to be tricky. Probably better to simply generate counts with trustworthy sources that use completely different techniques to generate their counts. For the smallest counts, we can verify the counts by adding a blocking clause for each model and verifying that the result is UNSAT.

As for diversity: the best way to trigger bugs is just to have counting problems from different sources/application domains. Things like number of variables, number of clauses, primal graph stats etc are of limited value here.

Current ideas for generating diverse instances:

- Bayesian net to **Dar02**, **SBK05**, **BLKM16**.
- DNF2CNF.
- ASP for hamiltonian paths and cliques.
- Incrementally construct formula and count with compiler.
- PB constraints (compute count with CPLEX?).

TODO

- ☐ Mate: Collect Model Counting competition binaries.
- ☐ Mate: Integrate them into fuzzer.

- ☐ Mate: Find DNF2CNF script.
- ☐ Anna: Find BN encoding scripts for **Dar02**, **SBK05** and **BLKM16**.
- ☐ both: write CNF file checker (header, numbers of clauses and variables, *etc.*)

Thu 9 Nov 2022: Anna & Mate

Input format: mccompetition.org/assets/files/2021/competition2021.pdf.

- Perturb networks from www.bnlearn.com/bnrepository.
- Use GRID and DQMR.
- Layer-by-layer DAG generation (inspired by CNFuzz).
- Fuzz projected weighted model counting vs weighted model counting

- ☐ Anna: PB constraints and CPLEX
- ☐ Mate: DNF2CNF
- ☐ Mate+Anna: random DAG construction for BNs
- ☐ Anna: fix wncf syntax to new input format

Low priority:

- ☐ Incrementally construct formula and count with compiler.
- ☐ PB constraints (compute count with CPLEX?).

Fri 18 Nov 2022: Anna & Mate

Mon 21 Nov 2022: Anna

Some thoughts on when to count a model count from ApproxMC4 as "wrong".

Given a CNF formula F, a tolerance $\epsilon > 0$ (typically set to 0.8) and a confidence δ (typically set to 0.1), ApproxMC4 returns a count c such that:

$$\Pr\left[\frac{|Sol(F)|}{1+\epsilon} \leq c \leq (1+\epsilon)\cdot|Sol(F)|\right] \geq 1-\delta \tag{1}$$

holds.

Let us assume that the true count is c^*, and that if we run ApproxMC4 on F repeatedly with different seeds, we will get a normal distribution $\mathcal{N}(c^*, \epsilon \cdot c^*)$, with mean $\mu_c = c^*$ and standard deviation $\sigma_c = c^* \cdot \epsilon$ as the number of samples approaches infinity.[Q2] [A]

[Q2] Is it reasonable to assume that $\epsilon \cdot c^*$ is a good standard deviation for this distribution?

[A] Maybe study Chebhyshev's and Paley-Zygmund inequalities in more detail to find an answer to that question?

We could run ApproxMC4, n times on F, with n different seeds, and record the counts $c_1, \ldots, c_n$. Let us call the resulting distribution $\mathcal{Q}$. We could then compute the *Kolmogorov-Smirnov statistic* between the CDF of $\mathcal{Q}$ and the CDF of $\mathcal{N}$. This statistic measures the largest absolute distance between the two distributions, and returns a p-value. We could choose the p-value for which we reject the null-hypothesis (both distributions are the same) maybe based on δ? Since ApproxMC4 tends to return the exact count if the counts are small, maybe we should focus on counts that exceed 1000 or something.

Tue 22 Nov 2022: Anna

Figures figs. 1 to 3 show some typical examples of the distributions that we're getting from sampling ApproxMC4. The plots show a histogram of the measured data, along with a fitted normal distribution and a vertical line to indicate the true count. In the title, I've added the fitted parameters (mu and std), and the true count (c*) and best guess for the std, computed as $\sigma = \frac{\epsilon}{1.65} * \mu_c$.

Clearly, the measured standard deviation is *much* smaller than the one obtained by computing it based on ϵ and δ.

We probably shouldn't think of this as looking at the mean and standard deviation, but rather think about the outliers. After all, we probably would not expect every run of ApproxMC4 to return a 'wrong' count, but only a few. So maybe, we have to visualise this using box plots, and come up with a way to define outliers? That's probably something to discuss for next time.

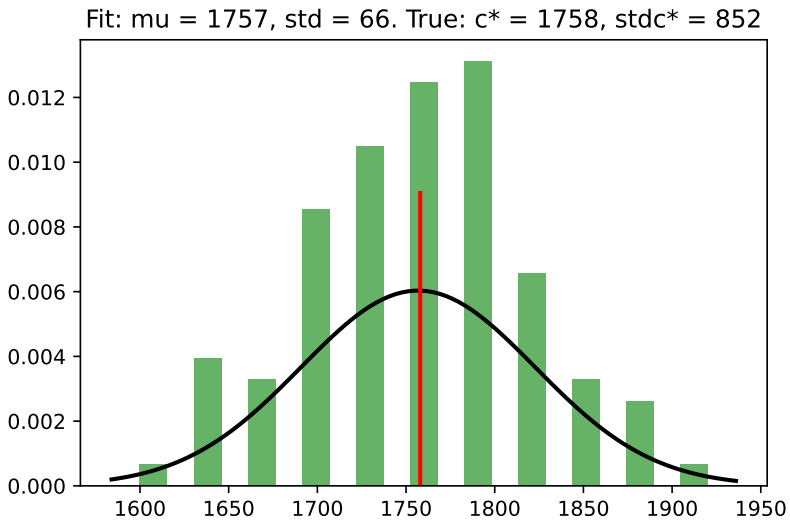

Figure 1: `sandbox/approxmc-results/nopreproc/fuzzTest-24.cr`

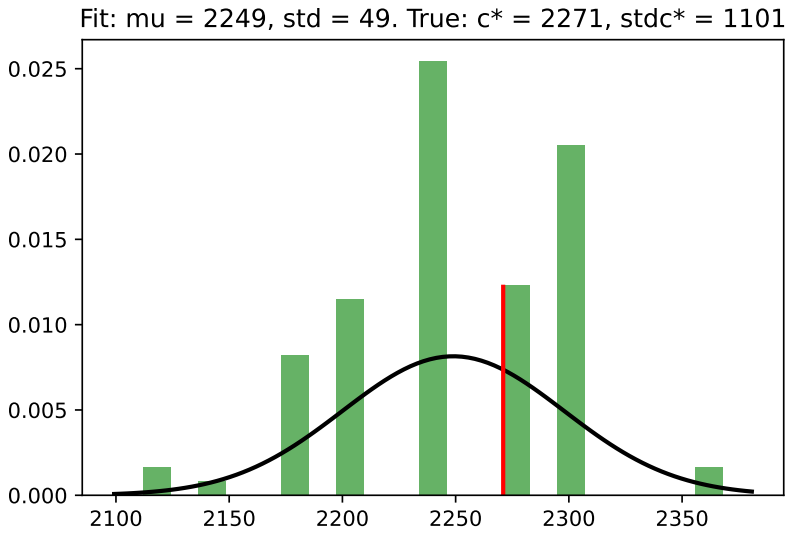

Figure 2: `sandbox/approxmc-results/nopreproc/fuzzTest-44.cr`

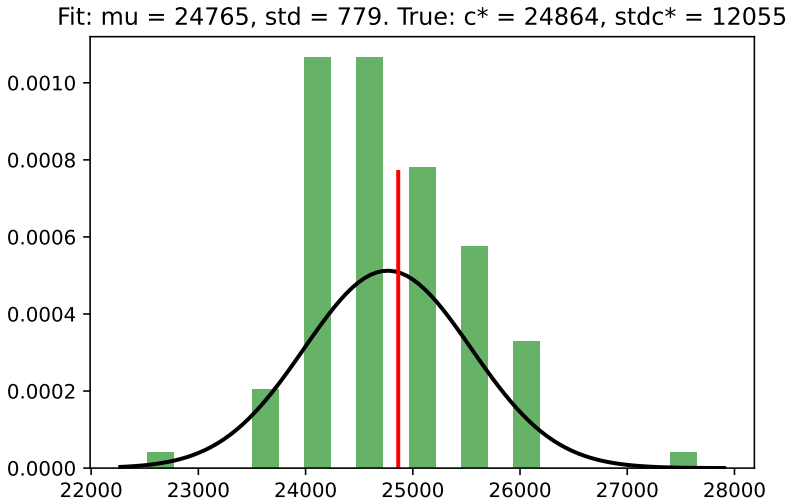

Figure 3: `sandbox/approxmc-results/nopreproc/fuzzTest-28.cr`

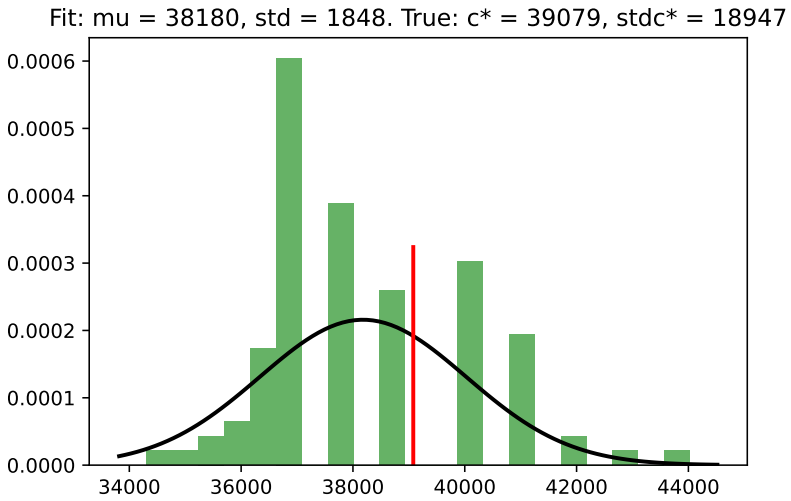

Figure 4: `sandbox/approxmc-results/nopreproc/fuzzTest-36.cr`

Thu 24 Nov 2022: Anna

Mate suggested to maybe not focus on figuring out when ApproxMC4 returns a wrong count, but to try to identify CNFs for which ApproxMC4 has a count distribution that is way off: either skewed and not Gaussian at all, or centered far away from the true count.

The Python packages that we use surely have some measure that we can use to quantify how skewed the distribution is. We can then use delta debugging to construct a CNF that skews the distribution as much as possible.

Thu 27 June 2024: Anna

I want to make a plan for creating and releasing a counter fuzzer, and for participating in writing the report for the yearly model counting competition.

Ideally, I would create a **fuzzer** that does the following things:

- Take as input:
  - one or more (exact or approximate) counters *of the same type* (cnf, wcnf, pcnf, wpcnf, pb, ...)
  - one or more problem generators suitable for that type
  - a maximum number of iterations
  - additional parameters for the evaluation, *e.g.*, time and space resources, confidence and precision parameters for approximate counting.
- Generate problem instances *with known count*, save those that cause a bug or miscount in at least one of the tested solvers.

- Return a short, human-readable and machine-parsable report on findings, and store findings.

In addition, I would create a set of **experimentation and evaluation scripts** that ideally generate some statistics and figures that can be included in the model counting competition reports.

Finally, I would create a **delta-debugger**, that would take as input a model counter and an instance on which the model counter fails, and try to minimise the instance to obtain an as-small-as-possible instance on which the model counter fails.

Tue 2 July 2024: Anna

Just as a reminder to myself, from now on, I will have to work with the following scripts/repos:

- `parse_counts_util.py` can be used to parse the standardised output of counters.
- `genrandomweights2.py` can be used for generating wCNFs from a counting graph.
- `genrandomweights.py` can be used to generate a random(?) wCNF.
- `step5aa_random_weights.py` can be used to generate wCNFs, pCNFs and pwCNFs.
- The competition scripts can be found here. It would be good if my code was run similarly.
- Johannes also suggested that I take inspiration from the benchmarking framework, which should also run on the cluster: copperbench.

Regarding the last point, Johannes writes:

> Comment on the cluster, if you want to do things the easy way for now, it might be an option to use a terminal multiplexer (screen/byobu) and leave it running in the background. Or if it's a slurm cluster login from a university desktop computer and get a node on the cluster `srun -n 1 -c NUMCORES -p PARTITION --pty bash` and run it there. (We do have a benchmarking framework for systematic evaluations: https://github.com/tlyphed/copperbench Should work on other clusters as well, can do a quick introduction at some point if you want to. But for a fuzzer, I would avoid the cluster.)

References

A. Bart, F. Koriche, J. Lagniez, and P. Marquis. An improved CNF encoding scheme for probabilistic inference. In G. A. Kaminka, M. Fox, P. Bouquet, E. Hüllermeier, V. Dignum, F. Dignum, and F. van Harmelen, editors, *ECAI 2016 - 22nd European Conference on Artificial Intelligence, 29 August-2 September 2016, The Hague, The Netherlands - Including Prestigious Applications of Artificial Intelligence (PAIS 2016)*, volume 285 of *Frontiers in Artificial Intelligence and Applications*, pages 613–621. IOS Press, 2016. DOI: 10.3233/978-1-61499-672-9-613. URL https://doi.org/10.3233/978-1-61499-672-9-613.

R. Brummayer and A. Biere. Fuzzing and delta-debugging smt solvers. In *Proceedings of the 7th International Workshop on Satisfiability Modulo Theories*, SMT 2009, pages 1–5, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605584843. DOI: 10.1145/1670412.1670413. URL https://doi.org/10.1145/1670412.1670413.

R. Brummayer, F. Lonsing, and A. Biere. Automated testing and debugging of SAT and QBF solvers. In O. Strichman and S. Szeider, editors, *Theory and Applications of Satisfiability Testing - SAT 2010, 13th International Conference, SAT 2010, Edinburgh, UK, July 11-14, 2010. Proceedings*, volume 6175 of *Lecture Notes in Computer Science*, pages 44–57. Springer, 2010. DOI: 10.1007/978-3-642-14186-7_6. URL https://doi.org/10.1007/978-3-642-14186-7_6.

S. Chakraborty, D. J. Fremont, K. S. Meel, S. A. Seshia, and M. Y. Vardi. Distribution-aware sampling and weighted model counting for SAT. In C. E. Brodley and P. Stone, editors, *Proceedings of the Twenty-Eighth AAAI Conference on Artificial Intelligence, July 27 -31, 2014, Québec City, Québec, Canada*, pages 1722–1730. AAAI Press, 2014. URL http://www.aaai.org/ocs/index.php/AAAI/AAAI14/paper/view/8364.

S. Chakraborty, K. S. Meel, and M. Y. Vardi. Algorithmic improvements in approximate counting for probabilistic inference: From linear to logarithmic SAT calls. In *Proceedings of International Joint Conference on Artificial Intelligence (IJCAI)*, 7 2016.

M. Chavira and A. Darwiche. Compiling Bayesian networks with local structure. In L. P. Kaelbling and A. Saffiotti, editors, *IJCAI-05, Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence, Edinburgh, Scotland, UK, July 30 - August 5, 2005*, pages 1306–1312. Professional Book Center, 2005. URL http://ijcai.org/Proceedings/05/Papers/0931.pdf.

M. Chavira and A. Darwiche. Encoding CNFs to empower component analysis. In A. Biere and C. P. Gomes, editors, *Theory and Applications of Satisfiability Testing - SAT 2006, 9th International Conference, Seattle, WA, USA, August 12-15, 2006,*

Proceedings, volume 4121 of *Lecture Notes in Computer Science*, pages 61–74. Springer, 2006. DOI: 10.1007/11814948_9. URL https://doi.org/10.1007/11814948_9.

A. Choi and A. Darwiche. Dynamic minimization of sentential decision diagrams. In M. desJardins and M. L. Littman, editors, *Proceedings of the Twenty-Seventh AAAI Conference on Artificial Intelligence, July 14-18, 2013, Bellevue, Washington, USA*. AAAI Press, 2013. URL http://www.aaai.org/ocs/index.php/AAAI/AAAI13/paper/view/6470.

A. Darwiche. A logical approach to factoring belief networks. In D. Fensel, F. Giunchiglia, D. L. McGuinness, and M. Williams, editors, *Proceedings of the Eights International Conference on Principles and Knowledge Representation and Reasoning (KR-02), Toulouse, France, April 22-25, 2002*, pages 409–420. Morgan Kaufmann, 2002.

A. Darwiche. New advances in compiling CNF into decomposable negation normal form. In R. L. de Mántaras and L. Saitta, editors, *Proceedings of the 16th Eureopean Conference on Artificial Intelligence, ECAI'2004, including Prestigious Applicants of Intelligent Systems, PAIS 2004, Valencia, Spain, August 22-27, 2004*, pages 328–332. IOS Press, 2004.

P. Dilkas and V. Belle. Weighted model counting without parameter variables. In C. Li and F. Manyà, editors, *Theory and Applications of Satisfiability Testing - SAT 2021 - 24th International Conference, Barcelona, Spain, July 5-9, 2021, Proceedings*, volume 12831 of *Lecture Notes in Computer Science*, pages 134–151. Springer, 2021a. DOI: 10.1007/978-3-030-80223-3_10. URL https://doi.org/10.1007/978-3-030-80223-3_10.

P. Dilkas and V. Belle. Weighted model counting with conditional weights for Bayesian networks. In C. P. de Campos, M. H. Maathuis, and E. Quaeghebeur, editors, *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence, UAI 2021, Virtual Event, 27-30 July 2021*, volume 161 of *Proceedings of Machine Learning Research*, pages 386–396. AUAI Press, 2021b. URL https://proceedings.mlr.press/v161/dilkas21a.html.

J. Dudek, V. Phan, and M. Vardi. Addmc: Weighted model counting with algebraic decision diagrams. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(02):1468–1476, Apr. 2020. DOI: 10.1609/aaai.v34i02.5505. URL https://ojs.aaai.org/index.php/AAAI/article/view/5505.

S. Ermon, C. Gomes, and B. Selman. Uniform solution sampling using a constraint solver as an oracle. In *Proceedings of the Twenty-Eighth Conference on Uncertainty in Artificial Intelligence*, UAI'12, page 255–264, Arlington, Virginia, USA, 2012. AUAI Press. ISBN 9780974903989.

J. K. Fichte, M. Hecher, and V. Roland. Proofs for propositional model counting. In K. S. Meel and O. Strichman, editors, *25th International Conference on Theory and Applications of Satisfiability Testing, SAT 2022, August 2-5, 2022, Haifa, Israel*, volume 236 of *LIPIcs*, pages 30:1–30:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022. DOI: 10.4230/LIPIcs.SAT.2022.30. URL https://doi.org/10.4230/LIPIcs.SAT.2022.30.

C. P. Gomes, J. Hoffmann, A. Sabharwal, and B. Selman. From sampling to model counting. In M. M. Veloso, editor, *IJCAI 2007, Proceedings of the 20th International Joint Conference on Artificial Intelligence, Hyderabad, India, January 6-12, 2007*, pages 2293–2299, 2007. URL http://ijcai.org/Proceedings/07/Papers/369.pdf.

V. Klebanov, A. Weigl, and J. Weisbarth. Sound probabilistic #sat with projection. In M. Tribastone and H. Wiklicky, editors, *Proceedings 14th International Workshop Quantitative Aspects of Programming Languages and Systems, QAPL 2016, Eindhoven, The Netherlands, April 2-3, 2016*, volume 227 of *EPTCS*, pages 15–29, 2016. DOI: 10.4204/EPTCS.227.2. URL https://doi.org/10.4204/EPTCS.227.2.

F. Koriche, J. Lagniez, P. Marquis, and S. Thomas. Knowledge compilation for model counting: Affine decision trees. In F. Rossi, editor, *IJCAI 2013, Proceedings of the 23rd International Joint Conference on Artificial Intelligence, Beijing, China, August 3-9, 2013*, pages 947–953. IJCAI/AAAI, 2013. URL http://www.aaai.org/ocs/index.php/IJCAI/IJCAI13/paper/view/6574.

J. Lagniez, E. Lonca, and P. Marquis. Improving model counting by leveraging definability. In S. Kambhampati, editor, *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, IJCAI 2016, New York, NY, USA, 9-15 July 2016*, pages 751–757. IJCAI/AAAI Press, 2016. URL http://www.ijcai.org/Abstract/16/112.

J. Lagniez, E. Lonca, and P. Marquis. Definability for model counting. *Artif. Intell.*, 281:103229, 2020. DOI: 10.1016/j.artint.2019.103229. URL https://doi.org/10.1016/j.artint.2019.103229.

Y. Lai, K. S. Meel, and R. Yap. The power of literal equivalence in model counting, 2021.

C. Muise, S. A. McIlraith, J. C. Beck, and E. Hsu. DSHARP: Fast d-DNNF Compilation with sharpSAT. In *Canadian Conference on Artificial Intelligence*, 2012.

U. Oztok and A. Darwiche. A top-down compiler for sentential decision diagrams. In Q. Yang and M. J. Wooldridge, editors, *Proceedings of the Twenty-Fourth International Joint Conference on*

Artificial Intelligence, IJCAI 2015, Buenos Aires, Argentina, July 25-31, 2015, pages 3141–3148. AAAI Press, 2015. URL http://ijcai.org/Abstract/15/443.

T. Sang, P. Beame, and H. A. Kautz. Heuristics for fast exact model counting. In F. Bacchus and T. Walsh, editors, *Theory and Applications of Satisfiability Testing, 8th International Conference, SAT 2005, St. Andrews, UK, June 19-23, 2005, Proceedings*, volume 3569 of *Lecture Notes in Computer Science*, pages 226–240. Springer, 2005a. DOI: 10.1007/11499107_17. URL https://doi.org/10.1007/11499107_17.

T. Sang, P. Beame, and H. A. Kautz. Performing Bayesian inference by weighted model counting. In M. M. Veloso and S. Kambhampati, editors, *Proceedings, The Twentieth National Conference on Artificial Intelligence and the Seventeenth Innovative Applications of Artificial Intelligence Conference, July 9-13, 2005, Pittsburgh, Pennsylvania, USA*, pages 475–482. AAAI Press / The MIT Press, 2005b. URL http://www.aaai.org/Library/AAAI/2005/aaai05-075.php.

S. Sharma, S. Roy, M. Soos, and K. S. Meel. GANAK: A scalable probabilistic exact model counter. In S. Kraus, editor, *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, pages 1169–1176. ijcai.org, 2019. DOI: 10.24963/ijcai.2019/163. URL https://doi.org/10.24963/ijcai.2019/163.

M. Soos and K. S. Meel. BIRD: Engineering an efficient CNF-XOR SAT solver and its applications to approximate model counting. In *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*, 1 2019.

M. Soos and K. S. Meel. Arjun: An efficient independent support computation technique and its applications to counting and sampling. *CoRR*, abs/2110.09026, 2021. URL https://arxiv.org/abs/2110.09026.

M. Soos, K. Nohl, and C. Castelluccia. Extending SAT solvers to cryptographic problems. In O. Kullmann, editor, *Theory and Applications of Satisfiability Testing - SAT 2009, 12th International Conference, SAT 2009, Swansea, UK, June 30 - July 3, 2009. Proceedings*, volume 5584 of *Lecture Notes in Computer Science*, pages 244–257. Springer, 2009. DOI: 10.1007/978-3-642-02777-2_24. URL https://doi.org/10.1007/978-3-642-02777-2_24.

M. Soos, S. Gocht, and K. S. Meel. Tinted, detached, and lazy CNF-XOR solving and its applications to counting and sampling. In S. K. Lahiri and C. Wang, editors, *Computer Aided Verification - 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21–24, 2020, Proceedings, Part I*, volume 12224 of *Lecture Notes in Computer Science*, pages 463–484. Springer, 2020. DOI: 10.1007/978-3-030-53288-8_22. URL https://doi.org/10.1007/978-3-030-53288-8_22.

S. Subbarayan, L. Bordeaux, and Y. Hamadi. Knowledge compilation properties of Tree-of-BDDs. In *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence, July 22-26, 2007, Vancouver, British Columbia, Canada*, pages 502–507. AAAI Press, 2007. URL http://www.aaai.org/Library/AAAI/2007/aaai07-079.php.

M. Thurley. sharpSAT — counting models with advanced component caching and implicit BCP. In A. Biere and C. P. Gomes, editors, *Theory and Applications of Satisfiability Testing - SAT 2006, 9th International Conference, Seattle, WA, USA, August 12-15, 2006, Proceedings*, volume 4121 of *Lecture Notes in Computer Science*, pages 424–429. Springer, 2006. DOI: 10.1007/11814948_38. URL https://doi.org/10.1007/11814948_38.

T. Toda and T. Soh. Implementing efficient all solutions sat solvers. *ACM J. Exp. Algorithmics*, 21, nov 2016. ISSN 1084-6654. DOI: 10.1145/2975585. URL https://doi.org/10.1145/2975585.

W. Wei and B. Selman. A new approach to model counting. In F. Bacchus and T. Walsh, editors, *Theory and Applications of Satisfiability Testing, 8th International Conference, SAT 2005, St. Andrews, UK, June 19-23, 2005, Proceedings*, volume 3569 of *Lecture Notes in Computer Science*, pages 324–339. Springer, 2005. DOI: 10.1007/11499107_24. URL https://doi.org/10.1007/11499107_24.

A. Zeller. *Why Programs Fail - A Guide to Systematic Debugging, 2nd Edition*. Academic Press, 2009. ISBN 978-0-12-374515-6. URL http://store.elsevier.com/product.jsp?isbn=9780123745156&pagename=search.